

HDOC DAY 32

PROBLEM

Write a C program to perform the following menu-driven operations using separate functions:

1. Get two variable length 2D arrays and prove that matrix addition is commutative.
2. Get two variable length 2D arrays and prove that matrix multiplication is not commutative.
3. Exit

C CODE

```
#include <stdio.h>

#include <stdlib.h>

/* Function declarations */

void matrixAddition();

void matrixMultiplication();

void inputMatrix(int **matrix, int rows, int cols);

void displayMatrix(int **matrix, int rows, int cols);

int **allocateMatrix(int rows, int cols);

void freeMatrix(int **matrix, int rows);

int main()
{
    int choice;

    do
    {
        printf("\n===== MENU =====\n");
```

```
printf("1. Matrix Addition - Prove Commutative Property\n");
printf("2. Matrix Multiplication - Prove Non-Commutative Property\n");
printf("3. Exit\n");
printf("=====\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
    case 1:
        matrixAddition();
        break;

    case 2:
        matrixMultiplication();
        break;

    case 3:
        printf("Program terminated.\n");
        break;

    default:
        printf("Invalid choice! Please try again.\n");
}

} while (choice != 3);

return 0;
```

```
}
```

```
/* Allocate memory for a 2D matrix */
```

```
int **allocateMatrix(int rows, int cols)
```

```
{
```

```
    int **matrix;
```

```
    int i;
```

```
    matrix = (int **)malloc(rows * sizeof(int *));
```

```
    for (i = 0; i < rows; i++)
```

```
    {
```

```
        matrix[i] = (int *)malloc(cols * sizeof(int));
```

```
    }
```

```
    return matrix;
```

```
}
```

```
/* Free memory allocated for a matrix */
```

```
void freeMatrix(int **matrix, int rows)
```

```
{
```

```
    int i;
```

```
    for (i = 0; i < rows; i++)
```

```
    {
```

```
        free(matrix[i]);
```

```
    }
```

```

    free(matrix);
}

/* Input matrix elements */
void inputMatrix(int **matrix, int rows, int cols)
{
    int i, j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            scanf("%d", &matrix[i][j]);
        }
    }
}

/* Display matrix */
void displayMatrix(int **matrix, int rows, int cols)
{
    int i, j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            printf("%d\t", matrix[i][j]);
        }
    }
}

```

```

        printf("\n");
    }
}

/* Matrix Addition: A + B = B + A */
void matrixAddition()
{
    int rows, cols;

    int i, j;

    printf("\nEnter number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    int **A = allocateMatrix(rows, cols);
    int **B = allocateMatrix(rows, cols);
    int **AB = allocateMatrix(rows, cols);
    int **BA = allocateMatrix(rows, cols);

    printf("Enter elements of Matrix A:\n");
    inputMatrix(A, rows, cols);

    printf("Enter elements of Matrix B:\n");
    inputMatrix(B, rows, cols);

    /* A + B and B + A */
    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)

```

```

    {
        AB[i][j] = A[i][j] + B[i][j];
        BA[i][j] = B[i][j] + A[i][j];
    }
}

printf("\nMatrix A:\n");
displayMatrix(A, rows, cols);

printf("\nMatrix B:\n");
displayMatrix(B, rows, cols);

printf("\nA + B:\n");
displayMatrix(AB, rows, cols);

printf("\nB + A:\n");
displayMatrix(BA, rows, cols);

/* Check commutativity */
int equal = 1;

for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        if (AB[i][j] != BA[i][j])
        {
            equal = 0;

```

```
        break;
    }
}
}
```

```
if (equal)
    printf("\nTherefore, A + B = B + A.\n");
else
    printf("\nA + B is not equal to B + A.\n");
```

```
freeMatrix(A, rows);
freeMatrix(B, rows);
freeMatrix(AB, rows);
freeMatrix(BA, rows);
}
```

```
/* Matrix Multiplication: A * B != B * A */
```

```
void matrixMultiplication()
{
    int r1, c1, r2, c2;
    int i, j, k;

    printf("\nEnter rows and columns of Matrix A: ");
    scanf("%d %d", &r1, &c1);

    printf("Enter rows and columns of Matrix B: ");
    scanf("%d %d", &r2, &c2);
```

```

/*
    To calculate both  $A*B$  and  $B*A$ ,
    matrices must have the same dimensions.

    Therefore, use square matrices.
*/

if (r1 != c1 || r2 != c2 || r1 != r2)
{
    printf("\nTo compare  $A*B$  and  $B*A$ , both matrices must be square ");
    printf("and of the same order.\n");
    return;
}

int n = r1;

int **A = allocateMatrix(n, n);
int **B = allocateMatrix(n, n);
int **AB = allocateMatrix(n, n);
int **BA = allocateMatrix(n, n);

printf("Enter elements of Matrix A:\n");
inputMatrix(A, n, n);

printf("Enter elements of Matrix B:\n");
inputMatrix(B, n, n);

/* Calculate  $A * B$  and  $B * A$  */
for (i = 0; i < n; i++)
{

```

```

    for (j = 0; j < n; j++)
    {
        AB[i][j] = 0;
        BA[i][j] = 0;

        for (k = 0; k < n; k++)
        {
            AB[i][j] += A[i][k] * B[k][j];
            BA[i][j] += B[i][k] * A[k][j];
        }
    }
}

```

```

printf("\nMatrix A:\n");
displayMatrix(A, n, n);

```

```

printf("\nMatrix B:\n");
displayMatrix(B, n, n);

```

```

printf("\nA * B:\n");
displayMatrix(AB, n, n);

```

```

printf("\nB * A:\n");
displayMatrix(BA, n, n);

```

```

/* Check whether multiplication is commutative */
int equal = 1;

```

```

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        if (AB[i][j] != BA[i][j])
        {
            equal = 0;
            break;
        }
    }
}

if (equal)
    printf("\nFor these matrices,  $A * B = B * A$ .\n");
else
    printf("\nTherefore,  $A * B \neq B * A$ .\n");

printf("Hence, matrix multiplication is not commutative in general.\n");

freeMatrix(A, n);
freeMatrix(B, n);
freeMatrix(AB, n);
freeMatrix(BA, n);
}

```